



Data & ML Engineer

Satellite Image Land-Use Classifier · BCA Data Science Capstone

P1 · You

P2 · Backend

P3 · Frontend

YOUR RESPONSIBILITIES

- ▶ Register Copernicus & Google Earth Engine accounts
- ▶ Set up Python environment, install all dependencies
- ▶ Download & explore EuroSAT dataset
- ▶ Fetch Sentinel-2 tiles using `sentinelsat`
- ▶ Write `preprocess.py` – band stacking (`B2,B3,B4,B8`)
- ▶ Compute NDVI & NDWI as extra channels (6-channel input)
- ▶ Build PyTorch `SatelliteDataset` class with transforms
- ▶ Write patch slicer (`64×64`, `32px` stride)
- ▶ Train baseline CNN (3 conv layers, ~75-80% accuracy)
- ▶ Fine-tune ResNet-50 with transfer learning (6-channel, 5 classes)
- ▶ Evaluate: confusion matrix, F1, IoU per class
- ▶ Save model weights → `models/resnet50-eurosat.pt`

YOUR TECH STACK

- ▶ Python 3.10+, PyTorch, torchvision (ResNet-50)
- ▶ `sentinelsat`, `earthengine-api`
- ▶ `rasterio`, GDAL, `numpy`, `scikit-learn`
- ▶ Google Colab (free T4 GPU for training)
- ▶ EuroSAT dataset (~2.8 GB)

Phase 1 – Setup & Exploration

Weeks 1-2

- 01 Register at `scihub.copernicus.eu` (Sentinel-2 API) and `earthengine.google.com` (use college email).
- 02 Create Python venv, install all packages. Install **GDAL before rasterio** (Ubuntu: `sudo apt install gdal-bin libgdal-dev`).
- 03 Download EuroSAT from Kaggle/HuggingFace. Open Jupyter, load patches with `rasterio`, visualise all 10 class types. Learn Band 4=Red, 8=NIR, 11=SWIR.
- 04 Use `sentinelsat` to download one cloud-free tile of chosen city (`cloudcoverpercentage=(0,10)`). Visualise true-color composite (`R=B04`, `G=B03`, `B=B02`).



Deliverable: Project proposal document – problem statement, city selection, expected outputs

Phase 2 – Data Preprocessing Pipeline

Weeks 3-5

- 01 Write **preprocess.py**: load Bands 2,3,4,8 from .SAFE folder, stack into 4-channel numpy array, normalise to [0,1] by dividing by 10000.
- 02 Compute **NDVI = (NIR-Red)/(NIR+Red)** and **NDWI = (Green-NIR)/(Green+NIR)** as channels 5 and 6.
- 03 Build **SatelliteDataset** in dataset.py: random flips, rotations, colour jitter. 70% train / 15% val / 15% test. DataLoader batch_size=32.
- 04 Write patch slicer: 64×64 window, 32px stride across full tile (~10,980×10,980px). Store patch coordinates for stitching (hand off to **P2**).



Deliverable: data_pipeline.ipynb – visualised patches, NDVI maps, DataLoader working end-to-end

Phase 3 – Model Training & Evaluation

Weeks 6-9

- 01 Build baseline CNN: 3 conv layers + maxpooling + 2 FC. Train 10 epochs on Colab. Target **~75-80% accuracy**. Document this baseline.
- 02 Fine-tune **ResNet-50**: modify first conv to 6-channel, freeze all except last 2 blocks + FC. Replace FC with nn.Linear(2048, 5). Train 20-30 epochs, AdamW + cosine LR.
- 03 Evaluate: confusion matrix (seaborn heatmap), per-class F1/precision/recall, **IoU per class**. Targets: overall accuracy >88%, water IoU >75%.
- 04 Run inference on real tile, stitch patch predictions → full classified map. Save as **GeoTIFF with correct CRS**. Pass weights (.pt) to P2.



Deliverable: models/resnet50_eurosat.pt + evaluation report with confusion matrix and F1 scores

TEAM CONTACTS

- P1** You – Data & ML Engineer
Training, preprocessing, model weights
- Person 2 – Backend & Inference
P2 Receives: patch coords, .pt weights → runs inference + change detection
[View P2 Tasks](#)
- Person 3 – Frontend & Deploy
P3 Receives: change stats CSV, classified maps → builds Streamlit UI
[View P3 Tasks](#)